

AIML Phase 2 Progress

Alexia-Madalina Cirstea, i6343775

Practical Steps:

- **Start small with training:** Pick an existing Unity ML-Agents environment (such as the Soccer Twos) and run the RL algorithms on it to see how it performs.
- **Custom sensor implementation:** Begin coding the sensor adjustments, such as changing how agents gather input in the Soccer Twos environment. Make sure it aligns with the project's goal of making the agents more realistic.
- **Document your progress:** Update GitHub repository with clear instructions on what has been changed and how to replicate your setup. This will help with the final evaluation and midway demonstration.

Task 1: RL training

Steps to Start Training in Unity ML-Agents (the plan I follow for the first task):

1. **Install ML-Agents and Setup the Environment:**
 - Clone the custom ML-Agents repository (the one mentioned in the project manual).
 - Open Unity, load the "Soccer Twos" environment or any pre-built ML-Agents environment.
 - Follow the installation steps to ensure ML-Agents are properly set up in Unity.
2. **Train Using the `mlagents-learn` Command:**
 - Once you have your Unity environment ready, use the `mlagents-learn` command in your terminal to start training the agents. You will need to point it to your environment and configuration file.

```
mlagents-learn config/ppo/SoccerTwos.yaml --env=SoccerTwos --run-id=soccer-run-1
```

Here:

- `config/ppo/SoccerTwos.yaml` is the path to the configuration file.
- `--env=SoccerTwos` points to the environment you want to train in.
- `--run-id=soccer-run-1` is the identifier for your training session.

Monitor the Training Progress:

- While training, ML-Agents provides several artifacts like **summary metrics** and **model checkpoints**.
- You can use **TensorBoard** to visualize the training progress and tune parameters accordingly.
 - To launch TensorBoard, use:

```
tensorboard --logdir results/
```

Adjust Training Parameters:

- The YAML configuration file (such as `SoccerTwos.yaml`) will contain various hyperparameters like learning rate, batch size, and reward signals.
- You can experiment with different configurations to see how they affect the agent's performance in training.

Stopping and Resuming Training:

- You can stop training using `Ctrl+C`, and it will save the model up to that point.
- To resume the training from where you left off, use the `--resume` flag when restarting training:

```
mlagents-learn config/ppo/SoccerTwos.yaml --env=SoccerTwos --run-id=soccer-run-1 --resume
```

Analyze Results:

- After the training is complete, analyze the agent's performance using TensorBoard and review the saved models and logs in the `results` folder.

Steps I took:

- 1) Open the terminal, activate the environment for the project (ml-agents, it's made in anaconda)
 - `conda activate ml-agents`
- 2) **Link Python with Unity:**
 - Ensure that the Python `mlagents-learn` tool is connected to the Unity environment.
 - In Unity, go to **Edit > Project Settings > Player**, and scroll down to find **Scripting Define Symbols**.

Add the following symbol:

`ML_AGENTS`

- Save and close the Project Settings window.

3) Steps to Open the Scene and Configure Agents:

1. **Open the SoccerTwos Scene:**
 - In the Inspector, click on the **"Open"** button you see in the screenshot to open the **SoccerTwos** scene.
 - Alternatively, go to the **Project** window (usually at the bottom of Unity) and double-click on the **SoccerTwos scene** file to open it.
2. **Find the Soccer Agents:**
 - After opening the scene, look for the agents in the **Hierarchy** window (typically on the left).
 - The agents are likely named something like **"SoccerAgent"** or similar.
3. **Select an Agent:**
 - Click on one of the agents in the **Hierarchy** window to display its properties in the **Inspector** window.
4. **Configure Behavior Parameters:**
 - With the agent selected, look in the **Inspector** window for a component called **Behavior Parameters**.
 - Ensure that the **Behavior Name** is set to match the name you are using in the Python YAML file for training, e.g., **SoccerTwos**.
 - If it's not there, you can add the **Behavior Parameters** component by clicking **Add Component** in the Inspector window and searching for "Behavior Parameters."
5. **Save the Scene:**
 - Once you've made sure everything is set up, save the scene by going to **File > Save**.

4) Install the mlagents in the python environment

5) try running the command again

I got this:

```
(ml-agents) C:\Users\User\Desktop\Project 2-1\ml-agents_AIML1\ml-agents>mlagents-learn
config/ppo/SoccerTwos.yaml --env=SoccerTwos --run-id=soccer-run-1
```

Traceback (most recent call last):

```
File "D:\Anaconda\envs\ml-agents\lib\runpy.py", line 196, in _run_module_as_main
```

```
    return _run_code(code, main_globals, None,
```

```
File "D:\Anaconda\envs\ml-agents\lib\runpy.py", line 86, in _run_code
```

```
    exec(code, run_globals)
```

```
File "D:\Anaconda\envs\ml-agents\Scripts\mlagents-learn.exe\__main__.py", line 4, in
```

```
<module>
```

```

File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents\trainers\learn.py", line 2, in
<module>
    from mlagents import torch_utils
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents\torch_utils\__init__.py", line 1, in
<module>
    from mlagents.torch_utils.torch import torch as torch # noqa
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents\torch_utils\torch.py", line 6, in
<module>
    from mlagents.trainers.settings import TorchSettings
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents\trainers\settings.py", line 25, in
<module>
    from mlagents.trainers.cli_utils import StoreConfigFile, DetectDefault, parser
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents\trainers\cli_utils.py", line 5, in
<module>
    from mlagents_envs.environment import UnityEnvironment
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents_envs\environment.py", line 12,
in <module>
    from mlagents_envs.side_channel.side_channel import SideChannel
File "D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents_envs\side_channel\__init__.py",
line 5, in <module>
    from mlagents_envs.side_channel.default_training_analytics_side_channel import ( # noqa
File
"D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents_envs\side_channel\default_training_a
nalytics_side_channel.py", line 7, in <module>
    from mlagents_envs.communicator_objects.training_analytics_pb2 import (
File
"D:\Anaconda\envs\ml-agents\lib\site-packages\mlagents_envs\communicator_objects\training_
analytics_pb2.py", line 35, in <module>
    _descriptor.FieldDescriptor(
File "D:\Anaconda\envs\ml-agents\lib\site-packages\google\protobuf\descriptor.py", line 621, in
__new__
    _message.Message._CheckCalledFromGeneratedFile()
TypeError: Descriptors cannot be created directly.
If this call came from a _pb2.py file, your generated code is out of date and must be
regenerated with protoc >= 3.19.0.
If you cannot immediately regenerate your protos, some other possible workarounds are:
1. Downgrade the protobuf package to 3.20.x or lower.
2. Set PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python (but this will use
pure-Python parsing and will be much slower).

```

More information:

<https://developers.google.com/protocol-buffers/docs/news/2022-05-06#python-updates>

!!!!Install python 3.8.x rather than python 3.10.x in both base and virtual env in anaconda

- Uninstall protobuf and install version 3.20.3 which is compatible with ml agents

Do the same for the following:

- catrs, numpy, pillow, grpcio, PyTorch, h5py

JUST REALIZED THAT WE DON'T HAVE A YAML FILE FOR SOCCERTWOS

- Made a yaml file
- Made an executable environment to run the trainer
- Ran the trainer
- Checking the tensorboard results

Install the tensorboard to see the results

Change the training from ppo to poca because of the multi-agent system

Trained them! See the first results i got here:

<http://localhost:6006/>

Now, let's change from ppo to poca in the yaml file

NOTE: USE THIS COMMAND TO TRAIN SOCCERTWOS AGENTS:

```
mlagents-learn "C:\Users\User\Desktop\Project
2-1\ml-agents_AIML1\config\ppo\SoccerTwos.yaml" --env="C:\Users\User\Desktop\Project
2-1\ml-agents_AIML1\config\ppo\UnityEnvironment" --run-id=soccer-run-X
```

Where X = number of the run to keep track

```
tensorboard --logdir="results/"
```

- To see the results

Progress of Task 1:

Experimenting with RL Algorithms:

- **Requirement:** Get comfortable with existing RL algorithms and explore their behavior in the "SoccerTwos" environment.
- **Progress:** Achieved this by testing PPO and switching to POCA, which is better suited for multi-agent scenarios. I have also been adjusting training parameters and observing results in TensorBoard.